

자율주행 출력 → 조인트 입력 수식 연결

자율주행 알고리즘의 출력이 각 바퀴/조향 조인트에 어떻게 전달되는지 수식적으로 정리한 문서.

기본 원리: 모든 조인트는 토크를 받는다

USD 조인트는 매 physics step마다 하나의 토크 값을 받아 회전합니다.
그 토크는 3개 입력의 합산으로 결정됩니다:

$$\begin{aligned}\tau_{\text{applied}} &= K_p \times (\theta_{\text{target}} - \theta_{\text{current}}) && \leftarrow \text{position term} \\ &+ K_d \times (\omega_{\text{target}} - \omega_{\text{current}}) && \leftarrow \text{velocity term} \\ &+ \tau_{\text{direct}} && \leftarrow \text{effort term}\end{aligned}$$

$$\tau_{\text{joint}} = \text{clamp}(\tau_{\text{applied}}, -\tau_{\text{limit}}, +\tau_{\text{limit}})$$

$$\begin{aligned}K_p &= \text{stiffness (Nm/rad)} \\ K_d &= \text{damping (Nm}\cdot\text{s/rad)} \\ \tau_{\text{limit}} &= \text{effort_limit_sim (Nm)}\end{aligned}$$

차량 조인트 구성 (TOY_01 기준)

차량 = 6개 조인트

조향 조인트 (2개):

front_left_steering_joint $\leftarrow$ revolute, 각도 제어
front_right_steering_joint $\leftarrow$ revolute, 각도 제어

구동 조인트 (2개, RWD):

rear_left_wheel_joint $\leftarrow$ revolute, 속도/토크 제어
rear_right_wheel_joint $\leftarrow$ revolute, 속도/토크 제어

프리 휠 (2개):

front_left_wheel_joint $\leftarrow$ revolute, 자유 회전
front_right_wheel_joint $\leftarrow$ revolute, 자유 회전

조향 조인트 수식

입력: 자율주행 → 조향각 (δ)

모든 자율주행 알고리즘은 가상 중앙 조향각 δ (rad) 를 출력합니다.

Case 1: 단순 조향 (좌우 동일)

$$\begin{aligned}\delta_{\text{left}} &= \delta \\ \delta_{\text{right}} &= \delta\end{aligned}$$

적용: `write_joint_position_to_sim( $\delta$ , joint_ids=[steer_left, steer_right])`

Case 2: Ackermann 기하학 (좌우 다름)

실제 차량은 내륜이 외륜보다 더 많이 꺾여야 합니다.

선회 반경:

$$R = L / \tan(|\delta|)$$

좌/우 바퀴 각도:

$$\begin{aligned}\delta_{\text{inner}} &= \text{atan}(L / (R - d/2)) && \leftarrow \text{안쪽 (더 많이 꺾임)} \\ \delta_{\text{outer}} &= \text{atan}(L / (R + d/2)) && \leftarrow \text{바깥쪽 (덜 꺾임)}\end{aligned}$$

L = wheelbase (축간 거리, m)

d = track_width (트랙 폭, m)

좌회전 ($\delta > 0$):

$$\delta_{\text{left}} = +\delta_{\text{inner}} = +\text{atan}(L / (R - d/2))$$

$$\delta_{\text{right}} = +\delta_{\text{outer}} = +\text{atan}(L / (R + d/2))$$

우회전 ($\delta < 0$):

$$\delta_{\text{left}} = -\delta_{\text{outer}} = -\text{atan}(L / (R + d/2))$$

$$\delta_{\text{right}} = -\delta_{\text{inner}} = -\text{atan}(L / (R - d/2))$$

적용:

`write_joint_position_to_sim( $\delta_{\text{left}}$ , joint_ids=[steer_left])`

`write_joint_position_to_sim( $\delta_{\text{right}}$ , joint_ids=[steer_right])`

Case 3: Extension AckermannController

Extension 내부에서 동일 계산을 수행:

입력: steeringAngle, wheelBase, trackWidth, speed

출력: wheelAngles[left, right], wheelRotationVelocity[4]

→ 우리가 직접 atan 계산할 필요 없음

조향 조인트의 PhysX 토크 계산

조향 조인트 설정: $K_p=8000$, $K_d=200$, $\tau_{\text{limit}}=500$

$\theta_{\text{target}} = \delta$ (자율주행 출력)

$\theta_{\text{current}} =$ 현재 바퀴 각도

$$\tau_{\text{steer}} = 8000 \times (\delta - \theta_{\text{current}}) + 200 \times (0 - \omega_{\text{current}})$$

$$\tau_{\text{steer}} = \text{clamp}(\tau_{\text{steer}}, -500, +500)$$

→ 바퀴가 δ 로 이동 (PD 제어)

또는 force_steer (우리 코드):

`write_joint_position_to_sim( $\delta$ )` → PD 무시, 즉시 각도 설정

구동 조인트 수식

자율주행 출력 → 바퀴 각속도/토크 변환

자율주행 출력 유형:

Type A: desired_velocity (m/s)

Type B: acceleration (m/s^2) + brake
Type C: throttle_percent (-1 ~ +1)

제어 방법 ① Velocity Target

자율주행: desired_velocity = v (m/s)

↓

단위 변환: $\omega_{\text{target}} = v / r$ (rad/s)

↓

PhysX: $\tau = K_d \times (\omega_{\text{target}} - \omega_{\text{current}})$

↓

바퀴에 적용

$r = \text{wheel_radius}$ (m)
 $K_d = \text{damping}$ ($\text{Nm} \cdot \text{s/rad}$)

예: $v=3 \text{ m/s}$, $r=0.0365 \text{ m}$
 $\omega_{\text{target}} = 3 / 0.0365 = 82.2 \text{ rad/s}$
현재 $\omega=0 \rightarrow \tau = 30 \times 82.2 = 2466 \text{ Nm} \rightarrow \text{clamp}(15) \rightarrow 15 \text{ Nm}$

적용:

`set_joint_velocity_target( $\omega_{\text{target}}$ )`
→ PhysX PD가 토크를 자동 계산

제어 방법 ② Effort (토크 직접)

자율주행: acceleration = a (m/s^2)

↓

뉴턴 법칙: $F = m \times a$ (N)

↓

토크 변환: $\tau = F \times r / n$ (Nm)

↓

바퀴에 적용

$m = \text{vehicle_mass}$ (kg)
 $r = \text{wheel_radius}$ (m)
 $n = \text{구동 바퀴 수}$ (RWD=2, 4WD=4)

예: $a=2 \text{ m/s}^2$, $m=2.5 \text{ kg}$, $r=0.0365 \text{ m}$, $n=2$
 $F = 2.5 \times 2 = 5.0 \text{ N}$
 $\tau = 5.0 \times 0.0365 / 2 = 0.091 \text{ Nm (per wheel)}$

적용:

`set_joint_effort_target( $\tau$ )`
→ PhysX에 직접 토크 전달 ($K_p=0$, $K_d=0$)

⚠ 속도 제한이 없어서 무한 가속 가능 → 코드에서 속도 리미터 필요:
`if current_speed > max_speed:`
`$\tau = 0$`

제어 방법 ②-B: Effort + 브레이크

자율주행: (acceleration, brake_percent)

↓

`if brake > 0:`

$$F_{\text{brake}} = -\text{brake} \times m \times g \times \mu_{\text{brake}} \quad (\text{N})$$

$$\tau = F_{\text{brake}} \times r / n$$

else:

$$F_{\text{drive}} = m \times \text{acceleration} \quad (\text{N})$$

$$\tau = F_{\text{drive}} \times r / n$$

$g = 9.81 \text{ m/s}^2$
 $\mu_{\text{brake}} = \text{브레이크 마찰계수} (\sim 0.8)$

적용:

$$\text{set_joint_effort_target}(\tau)$$

제어 방법 ②-C: Effort + 모터 모델 (VESC)

자율주행: $\text{throttle_percent} = t \text{ } (-1 \sim +1)$
 $\downarrow$
 모터 토크: $\tau_{\text{motor}} = t \times K_t \times I_{\text{max}} \quad (\text{Nm})$
 $\downarrow$
 기어비: $\tau_{\text{wheel}} = \tau_{\text{motor}} \times G \quad (\text{Nm})$
 $\downarrow$
 바퀴에 적용

$K_t = 60 / (2\pi \times K_V) \quad (\text{Nm/A, 토크 상수})$
 $I_{\text{max}} = \text{최대 전류 (A)}$
 $G = \text{gear_ratio}$

DC 모터 특성 (속도 올라가면 토크 감소):
 $\tau_{\text{motor}} = K_t \times (V_{\text{battery}} - K_t \times \omega_{\text{motor}} \times G) / R_{\text{motor}}$
 = 최대 토크에서 선형 감소

적용:

$$\text{set_joint_effort_target}(\tau_{\text{wheel}})$$

제어 방법 ③ Force Write Velocity

자율주행: $\text{desired_velocity} = v \text{ (m/s)}$
 $\downarrow$
 단위 변환: $\omega = v / r \quad (\text{rad/s})$
 $\downarrow$
 PhysX에 직접 속도 설정 (PD 무시, 무한 토크 가정)

적용:

$$\text{write_joint_velocity_to_sim}(\omega)$$

⚠ 비현실적 — 실차에서는 즉시 속도 변경 불가
 ✓ RL/프로토타입에 적합 — 간편하고 즉시 반응

제어 방법 ④ Force Write Position

자율주행: $\text{desired_velocity} = v \text{ (m/s)}$
 $\downarrow$
 각도 누적: $\theta(t) = \theta(t-1) + (v/r) \times dt \quad (\text{rad})$
 $\downarrow$
 PhysX에 직접 각도 설정

적용:

write_joint_position_to_sim(θ)

⚠ 각도를 매 step 누적해야 함

⚠ 실차와 안 맞음 (위치 제어 서보 모터가 아닌 이상)

제어 방법 ⑤ Ramp Velocity

자율주행: desired_velocity = v (m/s)

↓

Ramp: $v_cmd(t) = \min(v, v_cmd(t-1) + a_max \times dt)$ (m/s)

↓

단위 변환: $\omega = v_cmd / r$

↓

Force write

a_max = 최대 허용 가속도 (m/s²)

적용:

write_joint_velocity_to_sim(ω)

✓ ③보다 현실적 — 부드러운 출발, slip 감소

Extension 컨트롤러 수식

DifferentialController

입력: (V, ω) = (선속도 m/s, 각속도 rad/s)

출력: ($\omega_left, \omega_right$) = 좌/우 바퀴 각속도 (rad/s)

Unicycle 모델:

$$\omega_left = (2V - \omega \times b) / (2r)$$

$$\omega_right = (2V + \omega \times b) / (2r)$$

V = linear velocity (m/s)

ω = angular velocity (rad/s), 좌회전 양수

b = track width (△ cm 단위!)

r = wheel radius (△ cm 단위!)

자율주행 연결:

알고리즘 출력: (steering_angle δ , velocity v)

↓

Bicycle → Unicycle 변환:

$$V = v$$

$$\omega = v \times \tan(\delta) / L \quad (L = \text{wheelbase})$$

↓

DifferentialController.forward($[V, \omega]$)

↓

$[\omega_left, \omega_right]$

↓

write_joint_velocity_to_sim()

WheelBasePoseController

입력: (current_pos, current_yaw, goal_pos, goal_yaw)

출력: ($\omega_left, \omega_right$)

내부 로직:

1. heading_error = atan2(Δy , Δx) - current_yaw
2. distance = $\sqrt{(\Delta x^2 + \Delta y^2)}$

if distance < tol:

V=0, ω =0 → 정지

elif |heading_error| > heading_tol:

V=0, ω =yaw_velocity → 제자리 회전

else:

V=lateral_velocity, ω =0 → 직진

→ DifferentialController.forward([V, ω])

→ [ω_{left} , ω_{right}]

Stanley Control

입력: (현재 상태, 경로 cx/cy/cyaw)

출력: δ (조향각 rad)

수식:

1. nearest = argmin ||path[i] - pos|| (최근접 경로점)
2. heading_error = cyaw[nearest] - vehicle_yaw
3. crosstrack_error = 횡방향 거리
4. δ = heading_error + atan($k \times cte / (v + \epsilon)$)

k = Stanley gain (~0.5)

ϵ = 속도 0 방지 상수

출력 δ → 조향 조인트에 적용

PID Control

입력: (target_speed, current_speed)

출력: speed_command

수식:

cmd = $K_p \times (target - current)$

출력 → ω = cmd / r → 구동 조인트에 적용

Quintic Polynomial

입력: (시작 x,v,a) → (목표 x,v,a), 시간 T

출력: 경로점 배열 (x, y, yaw, v, a)

수식:

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

경계조건 6개:

$$x(0) = x_{start} \quad x(T) = x_{goal}$$

$$x'(0) = v_{start} \quad x'(T) = v_{goal}$$

$$x''(0) = a_{start} \quad x''(T) = a_{goal}$$

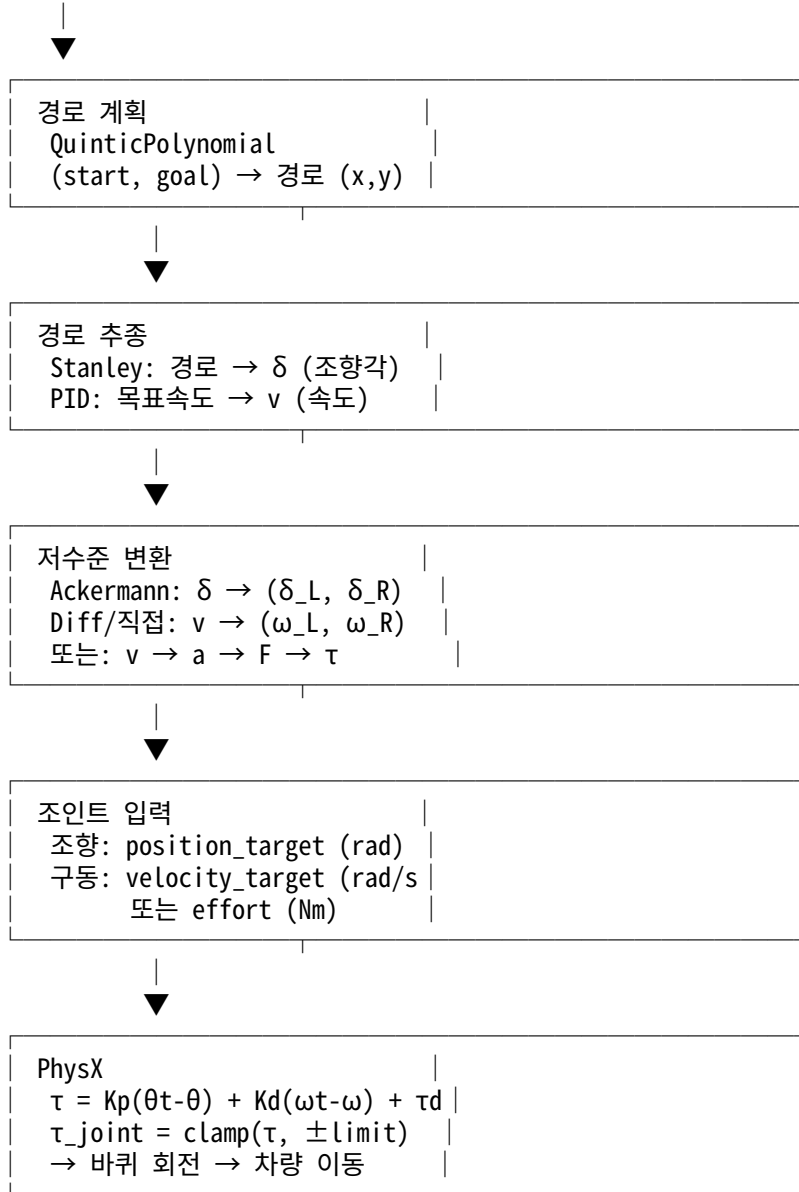
→ 6×6 연립방정식으로 계수 $a_0 \sim a_5$ 결정

→ jerk 연속성 보장 (3차 미분까지 존재)

출력 → Stanley + PID의 입력으로 사용

전체 파이프라인 요약

자율주행 스택



수치 예제 (TOY_01)

파라미터:

wheelbase $L = 0.2255$ m
track_width $d = 0.20$ m
wheel_radius $r = 0.0365$ m
mass $m = 2.5$ kg
max_steer = 0.6 rad (34.4°)

steer actuator: $K_p=8000$, $K_d=200$, $\tau_{\text{limit}}=500$
drive actuator: $K_p=0$, $K_d=30$, $\tau_{\text{limit}}=15$

예제 1: 직진 중 좌회전 (Velocity Control)

현재 상태:

차량 위치: (1.0, 0.0) 방향: 0° (동쪽)
chassis 속도: 2.0 m/s
후좌 바퀴 ω : 54.8 rad/s
후우 바퀴 ω : 54.8 rad/s
조향 각도: 0 rad

자율주행 출력:

steering_angle = +0.3 rad (좌회전 17.2°)
desired_velocity = 2.0 m/s (속도 유지)

Step 1: 조향 변환

단순: $\delta_{\text{left}} = \delta_{\text{right}} = 0.3$ rad

Ackermann:

$R = L / \tan(|\delta|) = 0.2255 / \tan(0.3) = 0.731$ m
 $\delta_{\text{left}} = \text{atan}(L / (R - d/2)) = \text{atan}(0.2255 / 0.631) = 0.343$ rad (19.7°)
 $\delta_{\text{right}} = \text{atan}(L / (R + d/2)) = \text{atan}(0.2255 / 0.831) = 0.265$ rad (15.2°)

Step 2: 조향 조인트에 적용

write_joint_position_to_sim:
front_left_steering_joint $\leftarrow$ 0.343 rad (즉시 설정)
front_right_steering_joint $\leftarrow$ 0.265 rad

또는 set_joint_position_target:

PhysX 계산: $\tau = 8000 \times (0.343 - 0.0) + 200 \times (0 - 0) = 2744$ Nm
clamp: $\min(2744, 500) = 500$ Nm $\rightarrow$ 바퀴가 0.343 rad으로 이동 시작
(수 step 후 도달)

Step 3: 구동 변환

$\omega_{\text{target}} = v / r = 2.0 / 0.0365 = 54.8$ rad/s
현재 $\omega = 54.8$ rad/s (이미 목표 속도)

Step 4: 구동 조인트에 적용

set_joint_velocity_target(54.8):
PhysX 계산: $\tau = 30 \times (54.8 - 54.8) = 0$ Nm
 $\rightarrow$ 현재 속도 유지 (추가 토크 불필요)

write_joint_velocity_to_sim(54.8):
 $\rightarrow$ PD 무시, 바퀴를 54.8 rad/s로 강제 유지

결과 (1 step 후):

차량이 좌측으로 커브 시작
선회 반경 $R = 0.731$ m

예제 2: 정지 상태에서 출발 (Effort Control)

현재 상태:

차량 위치: (0.0, 0.0) 방향: 90° (북쪽)
chassis 속도: 0 m/s
후좌 바퀴 ω : 0 rad/s
후우 바퀴 ω : 0 rad/s
조향 각도: 0 rad

자율주행 출력:

steering_angle = 0 rad (직진)
acceleration = 2.0 m/s² (출발)

Step 1: 조향

$\delta = 0 \rightarrow$ 조향 조인트 변화 없음

Step 2: 가속도 $\rightarrow$ 토크 변환

$F = m \times a = 2.5 \times 2.0 = 5.0$ N (필요한 힘)
 $\tau_{total} = F \times r = 5.0 \times 0.0365 = 0.183$ Nm (총 필요 토크)
 $\tau_{per_wheel} = 0.183 / 2 = 0.091$ Nm (바퀴당, RWD 2개)

Step 3: 구동 조인트에 적용

set_joint_effort_target(0.091):
rear_left_wheel_joint $\leftarrow \tau = 0.091$ Nm
rear_right_wheel_joint $\leftarrow \tau = 0.091$ Nm

PhysX (Kp=0, Kd=0):
 $\tau_{applied} = 0 + 0 + 0.091 = 0.091$ Nm
clamp: $0.091 < 15 \rightarrow$ 그대로 적용

결과 (매 step):

step 0: $\omega=0.0$, $v=0.00$ m/s, $\tau=0.091$ Nm
step 1: $\omega=2.5$, $v=0.09$ m/s, $\tau=0.091$ Nm (계속 가속)
step 2: $\omega=5.0$, $v=0.18$ m/s, $\tau=0.091$ Nm
...
step 100: $\omega=250$, $v=9.13$ m/s, $\tau=0.091$ Nm (⚠ 무한 가속!)

$\rightarrow$ 속도 리미터 필요:
if $v > \text{desired_max_speed}$:
 $\tau = 0$

예제 3: 고속 주행 중 감속 (Effort + Brake)

현재 상태:

차량 위치: (5.0, 3.0) 방향: 45°
chassis 속도: 4.0 m/s
후좌 바퀴 ω : 109.6 rad/s
후우 바퀴 ω : 109.6 rad/s
조향 각도: 0.1 rad

자율주행 출력:

steering_angle = 0.1 rad (유지)
acceleration = 0 m/s²
brake = 0.5 (50% 브레이크)

Step 1: 조향

$\delta = 0.1$ rad $\rightarrow$ 현재 0.1 $\rightarrow$ 변화 없음

Step 2: 브레이크 → 토크 변환

$$\begin{aligned} F_{\text{brake}} &= -\text{brake} \times m \times g \times \mu \\ &= -0.5 \times 2.5 \times 9.81 \times 0.8 \\ &= -9.81 \text{ N} \end{aligned}$$

$$\begin{aligned} \tau_{\text{per_wheel}} &= F_{\text{brake}} \times r / n \\ &= -9.81 \times 0.0365 / 2 \\ &= -0.179 \text{ Nm (음수 = 감속 방향)} \end{aligned}$$

Step 3: 구동 조인트에 적용

```
set_joint_effort_target(-0.179):  
    rear_left_wheel_joint ←  $\tau = -0.179 \text{ Nm}$   
    rear_right_wheel_joint ←  $\tau = -0.179 \text{ Nm}$ 
```

PhysX:
 $\tau_{\text{applied}} = 0 + 0 + (-0.179) = -0.179 \text{ Nm}$
바퀴 회전 감속

결과:

step 0: $\omega=109.6$, $v=4.00 \text{ m/s}$
step 50: $\omega=85.2$, $v=3.11 \text{ m/s}$
step 100: $\omega=60.8$, $v=2.22 \text{ m/s}$
...점차 감속

예제 4: DifferentialController로 제자리 회전

현재 상태:

차량 위치: (0.0, 0.0) 방향: 0°
chassis 속도: 0 m/s

자율주행 출력:

linear_velocity = 0 m/s (전진 없음)
angular_velocity = 2.0 rad/s (좌회전)

Step 1: DifferentialController 계산

$V = 0 \text{ m/s}$
 $\omega = 2.0 \text{ rad/s}$
 $b = 20 \text{ cm}$ (track width)
 $r = 3.65 \text{ cm}$ (wheel radius)

$$\begin{aligned} \omega_{\text{left}} &= (2 \times 0 - 2.0 \times 20) / (2 \times 3.65) = -40 / 7.3 = -5.48 \text{ rad/s} \\ \omega_{\text{right}} &= (2 \times 0 + 2.0 \times 20) / (2 \times 3.65) = +40 / 7.3 = +5.48 \text{ rad/s} \end{aligned}$$

Step 2: 구동 조인트에 적용

```
write_joint_velocity_to_sim:  
    rear_left_wheel_joint ←  $-5.48 \text{ rad/s}$  (후진 방향)  
    rear_right_wheel_joint ←  $+5.48 \text{ rad/s}$  (전진 방향)  
→ 좌바퀴 뒤로, 우바퀴 앞으로 → 제자리 좌회전
```

결과:

차량이 제자리에서 반시계 방향으로 회전
회전 속도 = $2.0 \text{ rad/s} \approx 114.6^\circ/\text{s}$
1초 후 약 115° 회전

예제 5: WheelBasePoseController로 목표점 이동

현재 상태:

차량 위치: (0.0, 0.0) 방향: 0° (동쪽)

목표:

goal_position = (2.0, 2.0)
goal_orientation = 45° (북동쪽)

Step 1: 컨트롤러 내부 계산

heading_to_goal = atan2(2.0 - 0.0, 2.0 - 0.0) = 45° = 0.785 rad
heading_error = 0.785 - 0.0 = 0.785 rad

|heading_error| = 0.785 > heading_tol (0.1)

→ 판단: "먼저 방향을 맞춰야 함"

→ $V = 0$, $\omega = \text{yaw_velocity} = 1.0 \text{ rad/s}$

Step 2: DifferentialController

$\omega_{\text{left}} = (0 - 1.0 \times 20) / (2 \times 3.65) = -2.74 \text{ rad/s}$

$\omega_{\text{right}} = (0 + 1.0 \times 20) / (2 \times 3.65) = +2.74 \text{ rad/s}$

→ 제자리 우회전 (45° 방향으로)

Step 3: 방향 맞춘 후

heading_error < 0.1 rad

distance = $\sqrt{4+4} = 2.83 \text{ m} > \text{position_tol} (0.1)$

→ 판단: "직진"

→ $V = \text{lateral_velocity} = 0.5 \text{ m/s}$, $\omega = 0$

$\omega_{\text{left}} = \omega_{\text{right}} = 0.5 / 0.0365 = 13.7 \text{ rad/s}$

→ 직진으로 목표점까지 이동

Step 4: 도착

distance < 0.1 m

→ $V = 0$, $\omega = 0$

→ 정지

예제 6: Stanley 경로 추종 중 횡방향 오차 보정

현재 상태:

차량 위치: (1.5, 0.3) 방향: 5° (약간 북쪽으로 틀어짐)

chassis 속도: 1.5 m/s

경로: y=0 직선 (동쪽으로)

Stanley 계산:

nearest path point: (1.5, 0.0)

crosstrack_error (cte) = 0.3 m (경로 북쪽으로 벗어남)

heading_error = 0° - 5° = -5° = -0.087 rad

speed = 1.5 m/s

k = 0.5 (Stanley gain)

$\delta = \text{heading_error} + \text{atan}(k \times \text{cte} / (\text{speed} + \epsilon))$
= -0.087 + atan(0.5 × 0.3 / (1.5 + 0.001))

= -0.087 + atan(0.1)
= -0.087 + 0.0997
= +0.013 rad (약간 우회전 → 경로로 복귀)

조인트 적용:

조향: write_joint_position_to_sim(0.013)
→ 거의 직진이지만 약간 우측으로 → 경로 복귀

구동: PID speed control
target=1.5, current=1.5 → $\text{cmd} = K_p \times 0 = 0$
→ 현재 속도 유지

$\omega = 1.5 / 0.0365 = 41.1 \text{ rad/s}$
write_joint_velocity_to_sim(41.1)

결과:

수 step 후: 차량이 $y=0$ 경로로 서서히 복귀
cte 감소: $0.3 \rightarrow 0.25 \rightarrow 0.18 \rightarrow \dots \rightarrow 0.0$